



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA
Campus Vitacura

Laboratorio 3: AndesHogar SpA

Autor: Iván González Vela

Semilla personal $S = 360$, $U = \$40.000$, $D = 7$

ICN292: Laboratorio

Profesor: José Luis Sáez Tamayo

Fecha: Septiembre 2026



Índice

1. Parámetros	2
2. Workflow de Triage	2
2.1. Reglas de clasificación	2
2.2. Flujo tras la clasificación	3
3. Workflow Emisor	3
4. Workflow Resumen Diario	3
5. Parte C	3
5.1. Configuración previa	3
5.2. C1	3
5.3. C3	4
5.4. C4	4
5.5. C5	4
6. Bonus	5
6.1. Comparación	5

Resumen ejecutivo

Se diseñó e implementó un sistema de triage automatizado para AndesHogar SpA usando la plataforma n8n. El sistema recibe solicitudes de garantía vía HTTP POST y las clasifica en cuatro rutas (DATOS_INVALIDOS, RECHAZO, REVISION, APROBACION) aplicando la política definida por los parámetros $S = 360$, $U = \$40.000$ y $D = 7$. Se desarrollaron tres workflows principales (triage, emisor y resumen diario) y un workflow bonus que reimplementa la clasificación con un único nodo de código. Se ejecutaron las pruebas de la Parte C, verificando el comportamiento ante entradas inválidas, casos límite, errores de tipo y casos irresolubles por diseño.

Para AndesHogar SpA se recomienda mantener el flujo principal basado en Switch y nodos Set. Esta implementación permite que cualquier operador ajuste las reglas directamente desde la interfaz visual de n8n, sin necesidad de escribir código, y ofrece un registro de ejecución nodo a nodo que facilita identificar qué condición se activó en cada solicitud. El nodo Code del bonus es más compacto y eficiente para iterar sobre la política, pero concentra toda la lógica en un punto único que requiere conocimiento de JavaScript para mantener o depurar, lo que representa una barrera para un equipo sin conocimiento técnico.

1. Parámetros

Parámetro	Descripción	Valor
S	Semilla personal	360
U	Umbral de monto máximo (CLP)	\$40.000
D	Días máximos desde compra	7

2. Workflow de Triage

El workflow ICN292-Lab3-Gonzalez-Ivan-triage recibe solicitudes POST en /trriage-dev y las clasifica mediante un nodo Switch con 6 reglas ordenadas más fallback. Consta de 11 nodos.

2.1. Reglas de clasificación

El nodo Switch opera en modo *Rules* con *Send data to all matching outputs* desactivado, de modo que solo aplica la primera regla que coincida.

Salida	Campo evaluado	Condición	Ruta asignada
0	id_solicitud	está vacío	DATOS_INVALIDOS
1	monto.toNumber()	≤ 0	DATOS_INVALIDOS
2	dias_desde_compra.toNumber()	> 7	RECHAZO
3	estado_producto	= danado_por_uso	RECHAZO
4	monto.toNumber()	$> 40,000$	REVISION
5	estado_producto	= con_fallas	REVISION
Fallback		ninguna regla anterior	APROBACION

Las reglas 1, 2 y 4 usan `.toNumber()` porque los campos llegan como texto desde el webhook, el tipo del operador se configura como **Number**.

2.2. Flujo tras la clasificación

Las rutas `DATOS_INVALIDOS`, `RECHAZO` y `REVISION` pasan directamente a los nodos de notificación y registro. La ruta `APROBACION` consulta adicionalmente la API `mindicador.cl/api` para obtener el valor de la UF vigente y agrega los campos `uf_valor` y `monto_en_uf` antes de continuar. Todos los caminos convergen en el nodo **Registrar** y terminan en **Respond to Webhook**.

3. Workflow Emisor

El workflow `ICN292-Lab3-Gonzalez-Ivan-emisor` genera y envía las 15 solicitudes de prueba al triage. Un **Manual Trigger** inicia la ejecución, un nodo **Code** genera el arreglo de 15 objetos JSON con combinaciones variadas de campos válidos e inválidos, y un **HTTP Request** envía cada uno al webhook del triage mediante POST.

4. Workflow Resumen Diario

El workflow `ICN292-Lab3-Gonzalez-Ivan-resumen` genera un reporte consolidado al cierre del día. Un **Schedule Trigger** llamado “Diario 20:00” lo activa todos los días a las 20:00, el nodo **Leer registro** lee los resultados del día, **Consolidar por ruta** agrupa y cuenta las solicitudes por ruta, **Armar mensaje** construye el texto del resumen, y **Notificacion resumen** envía el resultado al canal correspondiente.

5. Parte C

5.1. Configuración previa

El nodo **Clasificar** (Switch) se configuró con *On Error: Stop Workflow*, de modo que cualquier error en la evaluación de condiciones detiene el flujo y retorna un error HTTP al emisor.

5.2. C1

Caso	Entrada	Resultado observado
C1-a	<code>id_solicitud</code> vacío	Ruta <code>DATOS_INVALIDOS</code> , regla 0.
C1-b	<code>monto</code> ausente	HTTP 500: <i>toNumber can't be used on undefined value</i> . Flujo detenido.
C1-c	<code>monto = "0"</code>	Ruta <code>DATOS_INVALIDOS</code> , regla 1.

El caso C1-b es el comportamiento esperado: un campo obligatorio ausente provoca un error de tipo y el sistema lo rechaza de forma explícita.

5.3. C3

Caso	Entrada	Resultado observado
BORDE-U	monto = 40000 (igual a U)	Ruta APROBACION. La regla 4 es estricta ($> 40,000$), el valor exacto no la activa.
BORDE-D	dias_desde_compra = 7 (igual a D)	Ruta APROBACION. La regla 2 es estricta (> 7), el valor exacto no la activa.

Ambos casos consultaron `mindicador.cl` exitosamente y confirman que el límite exacto cae en APROBACION.

5.4. C4

Se cambió temporalmente la condición de la regla 4 de tipo **Number** a tipo **String**.
Error obtenido: *Wrong type: '9990' is a number but was expecting a string.*

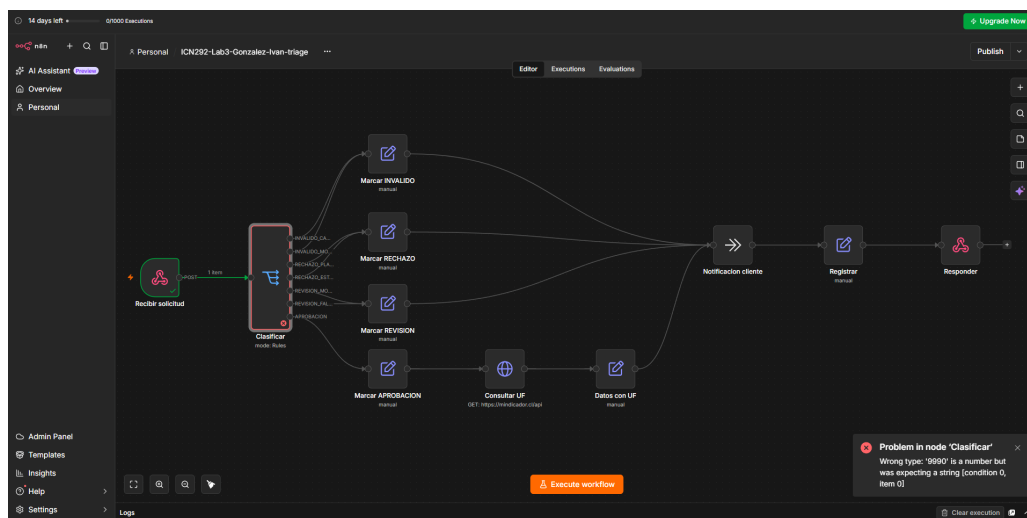


Figura 1: Error C4 en n8n: tipo incorrecto en condición del Switch.

n8n valida los tipos en tiempo de ejecución. Tras verificar el comportamiento, se restauró la condición a tipo **Number**.

5.5. C5

Caso	Entrada	Resultado observado
ERR-TEXT	monto = "veintemil"	HTTP 500: <i>cannot convert to number</i> . Flujo detenido.
ERR-SKU	sku = "ZZ-999" (inválido por negocio)	Ruta APROBACION. Cumple todas las condiciones formales de la política.

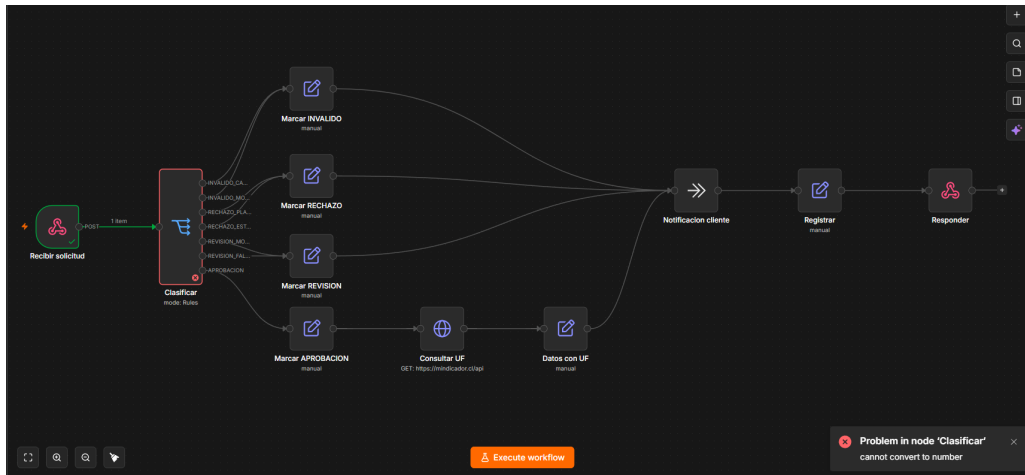


Figura 2: Error C5 en n8n: monto no convertible a número.

El caso ERR-SKU evidencia una limitación de diseño: la política no contempla validación de SKU contra un catálogo, por lo que el sistema no tiene forma de rechazarlo.

6. Bonus

6.1. Comparación

El flujo principal con Switch y nodos Set permite que un operador sin conocimientos de programación ajuste las reglas directamente desde la interfaz y ofrece un registro de ejecución nodo a nodo que facilita identificar qué condición se activó en cada solicitud. El nodo Code del bonus es más compacto y concentra toda la lógica en un solo lugar, lo que agiliza los cambios cuando la política se modifica con frecuencia, pero requiere JavaScript para mantener o revisar, lo que representa un riesgo para un equipo no técnico. Para AndesHogar SpA se recomienda el flujo principal; el bonus sería preferible solo si el equipo técnico asume el mantenimiento de los workflows.